Ergebnisse

XOR, 7-Segment-Decoder und ein lokales Minimum

Inhaltsverzeichnis

Interaktive Visualisierung 1
XOR 1
Was das Netz gelernt hat 2
Lokales Minimum 2
7-Segment-Decoder 2
Laufzeit 3
Export und Import 3
Austausch zwischen Python und C 4

Alle Zahlen stammen aus Läufen mit den Parametern der Angabe: Sigmoid, Lernrate $\alpha = 0,1$, 20 000 Epochen, Startgewichte gleichverteilt in $[-1, 1)$ mit festem Seed. Die Kosten sind der Mittelwert von J über alle Beispiele einer Epoche. Python und C# verwenden verschiedene Zufallsgeneratoren; derselbe Seed ergibt deshalb andere Startgewichte und einen etwas anderen Verlauf.

Interaktive Visualisierung

Wie sich Gewichte, Aktivierungen, Kosten und die Entscheidungsfläche beim XOR während des Trainings verändern, zeigt die interaktive Visualisierung mit allen drei Szenarien dieser Seite.

XOR

Netz 2-2-1, Seed 0 in beiden Sprachen.

Epoche	Kosten Python	Kosten C#
0	0,1315	0,1750
2 000	0,1253	0,1241
4 000	0,0991	0,0862
6 000	0,0100	0,0206
10 000	0,0017	0,0020
20 000	0,0005	0,0005

Die ersten rund 4 000 Epochen passiert wenig: Das Netz gibt für alle Inputs etwa 0,5 aus. Dann findet es die Lösung, und die Kosten fallen innerhalb weniger tausend Epochen um mehr als eine Zehnerpotenz.

Input	Soll	Ausgabe Python	Ausgabe C#	gerundet
[0, 0]	0	0,0343	0,0347	0 ✓
[0, 1]	1	0,9699	0,9690	1 ✓
[1, 0]	1	0,9708	0,9694	1 ✓
[1, 1]	0	0,0308	0,0325	0 ✓

Was das Netz gelernt hat

Die Gewichte des Python-Netzes (siehe Dateiformate) lassen sich direkt deuten:

Neuron	gewichtete Summe	aktiv bei
Hidden 1	$5,90\,x_1 - 5,91\,x_2 - 3,36$	$x_1 = 1$ und $x_2 = 0$
Hidden 2	$-5,42\,x_1 + 5,14\,x_2 - 2,86$	$x_1 = 0$ und $x_2 = 1$
Output	$8,16\,h_1 + 8,31\,h_2 - 4,06$	h_1 oder h_2 aktiv

Das Netz hat XOR also selbst in „(x_1 und nicht x_2) oder (x_2 und nicht x_1)„ zerlegt — genau die Form, die man auch von Hand aufschreiben würde.

Lokales Minimum

Ob das kleine XOR-Netz die Lösung findet, hängt von den Startgewichten ab. Mit den Seeds 0 bis 19 und sonst gleichen Parametern:

Ergebnis	Python	C#
alle 4 Fälle richtig, Kosten ≈ 0,0005	16 von 20	18 von 20
hängen geblieben, Kosten ≈ 0,064: zwei Ausgaben ≈ 0,5	Seeds 8, 12, 13	Seed 6
hängen geblieben, Kosten ≈ 0,085: drei Ausgaben ≈ 0,66	Seed 15	Seed 1

Bei Seed 8 lernt das Python-Netz $[0, 0] \to 0,03$ und $[0, 1] \to 0,96$, gibt aber für $[1, 0]$ und $[1, 1]$ beide Male etwa 0,50 aus. Eine Ausgabe von 0,5 ist für zwei Inputs mit unterschiedlichem Soll der beste Kompromiss, den dieses Netz von dort aus erreichen kann — der Gradient ist praktisch null, auch nach 20 000 Epochen. Das C#-Netz mit Seed 6 bleibt genauso hängen. Bei Seed 1 (C#) ist es die zweite Variante: Nur $[0, 0]$ ist richtig, die anderen drei Inputs landen alle bei etwa 0,66. In der Visualisierung ist das lokale Minimum als eigenes Szenario zu sehen.

Mehr Hidden-Neuronen, ein anderer Seed oder mehrere Läufe, von denen man den besten behält. Die XOR-Ergebnisse auf dieser Seite verwenden Seed 0.

7-Segment-Decoder

Netz 4-16-7, Seed 42 in beiden Sprachen. Alle **10 von 10 Ziffern** werden richtig dekodiert, in Python wie in C#.

Epoche	Kosten Python	Kosten C#
0	1,326	0,890
2 000	0,0129	0,0143
4 000	0,0040	0,0040
10 000	0,0011	0,0011
20 000	0,0005	0,0005

Mit 16 Hidden-Neuronen gibt es kein langes Plateau wie beim XOR; die Kosten fallen von Anfang an. Die Ausgaben der Python-Version:

Ziffer	Input	Soll a–g	Ausgabe a–g
0	0000	1111110	0,98 1,00 0,99 0,99 0,99 0,99 0,02
1	0001	0110000	0,02 1,00 1,00 0,01 0,00 0,02 0,00
2	0010	1101101	1,00 1,00 0,02 0,99 1,00 0,01 0,98
3	0011	1111001	1,00 0,99 0,99 0,99 0,01 0,00 0,98
4	0100	0110011	0,02 0,98 1,00 0,02 0,01 1,00 0,98
5	0101	1011011	0,98 0,02 1,00 0,98 0,00 0,99 0,98
6	0110	1011111	1,00 0,02 0,99 0,99 0,99 0,99 1,00
7	0111	1110000	0,98 0,99 1,00 0,02 0,00 0,00 0,03
8	1000	1111111	1,00 1,00 1,00 1,00 0,99 1,00 1,00
9	1001	1111011	0,99 0,99 1,00 1,00 0,00 0,99 0,99

Jede Ausgabe liegt höchstens 0,03 vom Sollwert entfernt, in der C#-Version höchstens etwa 0,02.

Laufzeit

Training über 20 000 Epochen	Python	C#
XOR 2-2-1	1,4 s	0,05 s
7-Segment 4-16-7	18 s	1,1 s

Beide Versionen rechnen Neuron für Neuron. Python ruft dabei für jedes Neuron numpy auf, und bei so kleinen Vektoren kostet jeder Aufruf mehr als die eigentliche Rechnung; C# übersetzt die Schleifen direkt in Maschinencode. Gemessen auf demselben PC, C# als Debug-Build, wie ihn `dotnet run` erzeugt.

Export und Import

Beide Netze wurden in beiden Sprachen als JSON, TXT und C-Header exportiert und aus JSON und TXT neu geladen. Die größte Abweichung zwischen Original und geladenem Netz über alle Inputs ist **0** — die Vorhersagen sind bitgleich.

Austausch zwischen Python und C

- C# lädt die in Python trainierten Netze (`dotnet run -- load ..\models\python\xor.json`) und gibt dieselben Ausgaben aus wie Python, beim XOR also 0,0343 / 0,9699 / 0,9708 / 0,0308. Der 7-Segment-Decoder aus Python erkennt auch in C# alle 10 Ziffern.
- Python lädt die in C# trainierten Netze genauso.
- Schreibt eine Sprache ein Netz der anderen neu, entsteht Byte für Byte dieselbe Datei — bei JSON, TXT und C-Header. Beide Testsammlungen prüfen das an den gespeicherten Modellen.